

CIQA - Pipeline Integration Guide

Adapting CIQA to any quantum compilation pipeline

Jessy Pensédent - arcadialab.fr

1. CIQA Output Format

The CIQA encoder is invoked as follows:

```
from CIQA import CIQA

ciqa = CIQA(n=5, d=2)
encoded = ciqa.encode(logical_circuit)
```

The `encode()` method returns a dictionary with four keys:

Key	Type	Description
<code>gates</code>	<code>list[dict]</code>	Ordered list of physical gate operations to execute.
<code>n_logical</code>	<code>int</code>	Number of logical qubits in the original circuit.
<code>n_physical</code>	<code>int</code>	Total physical qubits after encoding (<code>n_logical</code> × 5, plus ancillae).
<code>report</code>	<code>EncodingReport</code>	Encoding parameters. Pass downstream for auditing.

1.1 Gate Dictionary Format

Each entry in `gates` is a dictionary. Three gate types are produced:

Single-qubit rotation — Ry

```
{'type': 'Ry', 'qudit': 2, 'theta': 1.5707963267948966, 'd': 2}
```

`qudit` - physical qubit index (0-based).

`theta` - rotation angle in radians.

`'d': 2` - qudit dimension (always 2 for qubits).

Two-qubit controlled-X - CX

```
{'type': 'CX', 'control': 0, 'target': 1, 'd': 2}
```

`control` - index of the control qubit.

`target` - index of the target qubit.

Measurement - M

```
{'type': 'M', 'qubit': 3, 'd': 2, '_ciqa_physical': True}
```

`'_ciqa_physical': True` - marks a measurement introduced by CIQA (ancilla readout). Downstream pipelines may use this flag to distinguish CIQA measurements from user-circuit measurements.

The gate set is intentionally minimal: Ry and CX are universal for qubit computation. Any pipeline that supports single-qubit rotations and two-qubit controlled gates can accept CIQA output directly.

2. Integration Requirements

Two things are required to integrate CIQA into any pipeline:

2.1 A format conversion function

A function that iterates over `encoded['gates']` and translates each gate dictionary into the target pipeline's native gate representation. This is typically 15 to 40 lines of code.

2.2 A coupling map of sufficient size

The downstream pipeline's coupling map must have at least `n_physical` qubit positions, where `n_physical = encoded['n_physical']`. For CIQA `n=5`, this is `n_logical × 5` physical qubits plus ancillae. Pass a coupling map of the correct size to the downstream router; no other configuration change is needed.

CIQA itself does not perform routing or placement. Those stages remain entirely in the downstream pipeline. CIQA only expands the qubit count and prepends the encoding circuit.

3. Pipeline Adapters

The following adapters convert CIQA output to four common pipeline formats. Each is self-contained. Import CIQA, call `encode()`, then pass the result to the adapter function.

3.1 Qiskit

Requires: `pip install qiskit` (tested: `qiskit >= 1.0`)

```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister

def ciqa_to_qiskit(encoded: dict) -> QuantumCircuit:

    n = encoded["n_physical"]
    qr = QuantumRegister(n, "q")
    cr = ClassicalRegister(n, "c")
    qc = QuantumCircuit(qr, cr)

    for gate in encoded["gates"]:
        t = gate["type"]
        if t == "Ry":
            qc.ry(gate["theta"], qr[gate["qubit"]])
        elif t == "CX":
            qc.cx(qr[gate["control"]], qr[gate["target"]])
        elif t == "M":
            qc.measure(qr[gate["qubit"]], cr[gate["qubit"]])
        else:
            raise ValueError(f"Unknown gate type: {t}")

    return qc

# Usage
from CIQA import CIQA

ciqa = CIQA(n=5, d=2)
encoded = ciqa.encode(my_logical_circuit)
qc = ciqa_to_qiskit(encoded)

# Hand off to Qiskit transpiler
from qiskit import transpile
compiled = transpile(qc, coupling_map=my_coupling_map, basis_gates=["ry",
"cx"])
```

Qiskit note: pass `basis_gates=["ry", "cx"]` to the transpiler to prevent it from decomposing CIQA gates into other gate sets. CIQA encoding angles are exact fixed values; unnecessary decomposition introduces floating-point error.

3.2 OpenQASM 3

OpenQASM 3 is hardware-independent. Use this adapter when the target hardware accepts QASM directly, or when interoperating with tools like Stim, Cirq, or tket via QASM import.

```
def ciqa_to_openqasm3(encoded: dict) -> str:

    n = encoded["n_physical"]
    lines = [
        "OPENQASM 3.0;",
        "include \"stdgates.inc\";",
        "",
        f"qubit[{n}] q;",
        f"bit[{n}] c;",
        "",
    ]

    for gate in encoded["gates"]:
        t = gate["type"]
        if t == "Ry":
            theta = gate["theta"]
            q_idx = gate["qudit"]
            lines.append(f"ry({theta:.16f}) q[{q_idx}];")
        elif t == "CX":
            ctrl = gate["control"]
            tgt = gate["target"]
            lines.append(f"cx q[{ctrl}], q[{tgt}];")
        elif t == "M":
            q_idx = gate["qudit"]
            lines.append(f"c[{q_idx}] = measure q[{q_idx}];")
        else:
            raise ValueError(f"Unknown gate type: {t}")

    return "\n".join(lines)

# Usage
from CIQA import CIQA

ciqa = CIQA(n=5, d=2)
encoded = ciqa.encode(my_logical_circuit)
qasm = ciqa_to_openqasm3(encoded)

with open("encoded_circuit.qasm", "w") as f:
    f.write(qasm)
```

3.3 Generic Gate List

A pipeline-agnostic format. Use this as the starting point for any pipeline not listed above, or as an interchange format between tools.

```
def ciqa_to_generic(encoded: dict) -> dict:

    report = encoded.get("report")
    gates_out = []

    for gate in encoded["gates"]:
        t = gate["type"]
        is_ciqa = gate.get("_ciqa_physical", False)

        if t == "Ry":
            "name":      "RY",
            "qubits":    [gate["qudit"]],
            "params":    [gate["theta"]],
            "is_ciqa":   is_ciqa,
        })
        elif t == "CX":
            "name":      "CNOT",
            "qubits":    [gate["control"], gate["target"]],
            "params":    [],
            "is_ciqa":   is_ciqa,
        })
        elif t == "M":
            "name":      "MEASURE",
            "qubits":    [gate["qudit"]],
            "params":    [],
            "classical": gate["qudit"],
            "is_ciqa":   is_ciqa,
        })
        else:
            raise ValueError(f"Unknown CIQA gate type: {t}")

    return {
        "n_qubits": encoded["n_physical"],
        "n_logical": encoded["n_logical"],
        "gates":     gates_out,
        "metadata": {
            "n_physical": encoded["n_physical"],
            "n_logical":  encoded["n_logical"],
            "encoding":    "[[5,1,3]] perfect quantum code",
            "overhead_ratio": "1:5",
        },
    }
```

```
# Usage
from CIQA import CIQA

ciqa = CIQA(n=5, d=2)
encoded = ciqa.encode(my_logical_circuit)
generic = ciqa_to_generic(encoded)

# Access gates
for gate in generic["gates"]:
    print(gate["name"], gate["qubits"], gate.get("params", []))
```

4. One Constraint: Do Not Simplify Encoding Gates

Some compilation pipelines apply gate fusion or small-angle removal before routing. This is safe for user circuit gates but must not be applied to CIQA encoding gates.

CIQA encoding angles are exact fixed values. Removing or approximating them breaks the encoding and eliminates the error correction guarantee. The `_ciqa_physical` flag on measurement gates identifies CIQA-inserted operations; encoding gates (Ry and CX) do not carry this flag but appear before the user circuit gates in the list.

The safest approach: run CIQA first, then pass the encoded circuit directly to the router without a pre-routing simplification pass. Apply gate optimization only after routing, and only to gates that are not part of the encoding prefix.

If you are using CIQO as your optimizer, this is handled automatically. CIQO is aware of the encoding prefix and will not remove encoding gates. For external optimizers, protect the encoding prefix by splitting the circuit at the boundary between encoding gates and user-circuit gates.

5. Coupling Map Sizing Reference

The table below gives the minimum coupling map size for common circuit configurations using CIQA n=5:

Logical qubits	Physical qubits (n×5)	Ancillaes (x2)	Total minimum
1	5	2	7
10	50	20	70
100	500	200	700
1000	5000	2000	7000

In practice, the downstream router may require additional qubits for SWAP insertion depending on the hardware topology. The values above are the minimum for CIQA encoding only.

6. Quick Reference

Pipeline	Adapter function	Output type
Qiskit	<code>ciqa_to_qiskit()</code>	<code>qiskit.QuantumCircuit</code>
OpenQASM 3	<code>ciqa_to_openqasm3()</code>	<code>str</code>
Generic	<code>ciqa_to_generic()</code>	<code>dict {'n_qubits', 'gates', 'metadata'}</code>